

Sistemas Distribuídos — Aula 28

Laboratório: Desenvolvimento do Trabalho Final (Acompanhamento) · Prof. Leandro Borges

Por que uma aula de laboratório agora

Depois da Aula 27, dedicada à revisão geral do 2º bimestre e à orientação do trabalho final, a disciplina entra em uma reta final essencialmente prática. A Aula 28, marcada para 01/12, não traz conteúdo teórico novo: é uma aula de laboratório, reservada inteiramente para que cada grupo avance no Checkpoint 3 do trabalho evolutivo — a etapa de tolerância a falhas — com o professor circulando pela sala para apoiar decisões de implementação em tempo real, em vez de deixar essas dúvidas acumularem até a véspera da entrega.

O motivo para reservar uma aula inteira a isso é simples: o Checkpoint 3 é, ao mesmo tempo, a etapa de maior peso na rubrica de avaliação do trabalho (35% da nota) e a tecnicamente mais exigente das três, porque envolve simular e tratar falhas — algo que a maioria dos grupos nunca implementou antes. Um espaço estruturado em sala, com apoio direto do professor, ajuda a identificar cedo quais grupos estão no caminho certo e quais precisam de mais atenção, com tempo real de reação antes da apresentação parcial da Aula 29 e da entrega final na Aula 31, em 15/12.

Checklist de progresso: onde cada grupo deveria estar

Vale relembrar rapidamente os três checkpoints do trabalho evolutivo e o que se espera de cada grupo até esta aula. O Checkpoint 1 — comunicação cliente-servidor via sockets ou RPC/RMI — foi entregue em 08/09 e deveria estar plenamente concluído e estável: é a base sobre a qual tudo o resto é construído, e qualquer instabilidade nele tende a se propagar para as etapas seguintes. O Checkpoint 2 — evolução do sistema para múltiplas réplicas do servidor, com discussão do modelo de consistência adotado — foi entregue em 29/10 e também deveria estar concluído, já incorporado ao sistema em funcionamento.

O Checkpoint 3 — tratamento de falhas, seja por detecção de nó fora do ar, mecanismo de failover ou um esquema simples de consenso — é o que está em andamento nesta fase do semestre, com entrega final marcada para 15/12. Até esta aula, espera-se que cada grupo já tenha ao menos esboçado uma forma de detectar que um nó parou de responder, mesmo que o comportamento do sistema diante dessa falha ainda não esteja completo. Além do código, o relatório final — que deve reunir o histórico do projeto, as decisões de arquitetura tomadas ao longo das três etapas e os testes realizados — merece ser iniciado já nesta aula, e não deixado inteiramente para os últimos dias antes da entrega.

Dúvida comum: sockets crus ou um framework de RPC/gRPC?

Uma das dúvidas mais recorrentes nesta fase do projeto é se vale a pena trocar a tecnologia de comunicação já usada — em geral, sockets crus implementados no Checkpoint 1 — por um framework de RPC estruturado, como gRPC, para facilitar a implementação da tolerância a falhas. A resposta depende do estado atual do sistema de cada grupo, mais do que de qual tecnologia é objetivamente "melhor".

Se a base de sockets já está funcionando de forma estável, a recomendação geral é mantê-la: trocar de tecnologia de comunicação nesta altura do semestre custa tempo de reimplementação e reteste que poderia ser investido diretamente na tolerância a falhas, que é o que está sendo avaliado no Checkpoint 3. Por outro lado, se a comunicação do grupo ainda é instável ou pouco organizada, um framework como gRPC (com Protocol Buffers)

pode reduzir bastante o código repetitivo de serialização e oferece suporte nativo a streaming, o que facilita cenários de replicação e detecção de falha por heartbeat. O critério prático para decidir, portanto, não é qual tecnologia é mais moderna, e sim qual caminho o grupo consegue evoluir com segurança até 15/12.

Dúvida comum: como simular falhas e demonstrar consenso

A segunda dúvida técnica mais frequente nesta fase é de ordem prática: como simular a falha de um nó de forma controlada, para testar se o mecanismo de tolerância a falhas realmente funciona. Algumas formas simples e honestas de fazer isso incluem encerrar abruptamente o processo do servidor (por exemplo, com um comando equivalente a `kill -9`, ou simplesmente fechando o terminal em que ele roda), derrubar a interface de rede da máquina virtual ou contêiner que hospeda aquele nó, ou inserir deliberadamente no código um comando de pausa ou encerramento para um teste reproduzível. O ponto importante, em qualquer uma dessas abordagens, é que a falha simulada seja abrupta — um nó que simplesmente para de responder — e não um desligamento gracioso, que não representa o tipo de falha que o mecanismo do grupo precisa detectar.

Uma dúvida relacionada é como tornar visível, numa apresentação, algo que normalmente acontece nos bastidores do sistema, como uma eleição de líder ou uma decisão de consenso. A técnica mais eficaz é registrar em log cada etapa relevante da decisão — proposta, votos recebidos, valor aceito — no terminal de cada réplica, e depois mostrar esses terminais lado a lado durante a demonstração, para que a plateia acompanhe o processo em tempo real. Como apresentações ao vivo sempre correm o risco de algo dar errado no momento errado, um vídeo curto de tela gravado com antecedência funciona como evidência de apoio, garantindo que o grupo tenha algo para mostrar mesmo que a demonstração ao vivo falhe.

Por fim, vale esclarecer o que conta como uma implementação mínima aceitável de tolerância a falhas para efeito do Checkpoint 3: não é necessário implementar um algoritmo de consenso completo como Raft ou Paxos. O essencial é que o sistema seja capaz de detectar que um nó parou de responder — por exemplo, através de um mecanismo de timeout ou heartbeat — e que, a partir dessa detecção, o sistema continue operando de alguma forma ou se recupere. Um esquema de failover simples, desde que bem implementado e bem justificado no relatório, atende plenamente ao que o enunciado do trabalho pede.

Boas práticas de desenvolvimento incremental

Três hábitos simples de desenvolvimento separam, historicamente, os grupos que chegam à apresentação final em dia dos grupos que correm atrás na última semana. O primeiro é o versionamento disciplinado com Git: commits pequenos e frequentes, cada um correspondendo a uma funcionalidade específica, em vez de um único commit gigante no fim do processo — o que, além de dificultar a identificação de onde um problema foi introduzido, também é a forma mais simples de demonstrar, no histórico do repositório, que o trabalho foi de fato incremental e não montado às pressas. Trabalhar com branches por integrante ou por funcionalidade, com merges regulares, também evita que o trabalho de um membro do grupo sobrescreva o de outro.

O segundo hábito é testar cada parte do sistema à medida que ela é implementada, em vez de deixar todo o teste para o final. Cada novo mecanismo — detecção de falha, failover, qualquer forma de consenso — deveria ter, antes de o grupo seguir para a próxima etapa, ao menos um teste manual reproduzível que comprove que ele funciona como esperado. Essa prática torna muito mais simples identificar, mais tarde, exatamente qual mudança quebrou o quê, em vez de tentar depurar um sistema inteiro de uma vez só na véspera da apresentação.

O terceiro hábito é documentar as decisões de arquitetura no momento em que elas são tomadas, seja num README do repositório ou diretamente no rascunho do relatório final. Anotar, no dia em que uma escolha é feita, por que o grupo optou por uma determinada estratégia de replicação ou de detecção de falha — e quais alternativas foram descartadas e por quê — é muito mais confiável do que tentar reconstruir esse raciocínio de memória depois, quando o relatório final precisa ser fechado. Um sistema desenvolvido com esses três hábitos combinados normalmente chega à Aula 29 em condições de ser demonstrado com confiança, mesmo que ainda incompleto.

Como esta aula está organizada em sala

O tempo desta aula segue uma dinâmica simples, pensada para equilibrar acompanhamento individual e aproveitamento coletivo. Nos primeiros minutos, cada grupo faz um check-in rápido, de cerca de um minuto, resumindo em que ponto está no Checkpoint 3 e qual é sua principal dúvida no momento — isso ajuda o professor a priorizar quais grupos precisam de mais atenção ao longo da aula. Na sequência, a maior parte do tempo é dedicada ao desenvolvimento efetivo em grupo, com o professor circulando pela sala e parando alguns minutos em cada grupo conforme a necessidade identificada no check-in.

Perto do fim da aula, há uma rodada de dúvidas coletivas: questões técnicas que mais de um grupo levantou durante o desenvolvimento são resolvidas em conjunto, para que todos os grupos se beneficiem da discussão, e não só quem perguntou primeiro. A aula se encerra com um fechamento breve, em que cada grupo registra por escrito qual é o próximo passo concreto até a Aula 29 — um compromisso simples, sem burocracia, que ajuda a manter o ritmo de desenvolvimento entre uma aula e outra.

Preparando a Aula 29: apresentação parcial e dúvidas

A Aula 29, em 03/12, é uma apresentação parcial: cada grupo mostra, em um espaço de tempo curto, o progresso alcançado no Checkpoint 3 até aquele momento. É importante deixar claro que o sistema não precisa estar terminado nessa data — o Checkpoint 3 só tem entrega definitiva em 15/12, na Aula 31 —, mas precisa estar em um estado demonstrável, com algo concreto para mostrar em vez de apenas descrever verbalmente o que está planejado.

Para a Aula 29, cada grupo deveria chegar preparado para mostrar quatro coisas. Primeiro, que o sistema base — os Checkpoints 1 e 2, comunicação cliente-servidor e replicação — continua rodando de ponta a ponta, sem que o desenvolvimento do Checkpoint 3 tenha introduzido alguma regressão. Segundo, ao menos uma demonstração, ao vivo ou em vídeo de apoio, de um nó falhando e do sistema reagindo de alguma forma a essa falha. Terceiro, uma explicação curta — uma ou duas frases bastam — sobre qual estratégia de tolerância a falhas o grupo escolheu e por que essa foi a escolha. E quarto, uma lista honesta das pendências que ainda restam até a entrega final: parte do que é avaliado nessa etapa é justamente a capacidade do grupo de diagnosticar com clareza o próprio progresso, e não só a quantidade de código já escrita.

Próxima aula

A Aula 29 (03/12) é a apresentação parcial dos trabalhos: cada grupo apresenta brevemente o progresso do Checkpoint 3, seguida de uma rodada aberta de dúvidas antes da reta final. Depois dela, a Aula 30 (10/12) é uma revisão geral de todo o semestre, e a Aula 31 (15/12) é a apresentação final dos trabalhos, quando o Checkpoint 3 completo é entregue junto com o relatório final e cada grupo apresenta o projeto em até 10 minutos.

Referências

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. 2ª ed. São Paulo: Pearson Prentice Hall, 2008. Referência de base para tolerância a falhas, replicação e consenso, temas do Checkpoint 3.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T. Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman. Referência de apoio oficial do briefing do trabalho final.

gRPC Authors. gRPC Documentation. Disponível em: grpc.io/docs. Referência prática para grupos que optarem por RPC estruturado em vez de sockets crus.

Git Documentation. Pro Git Book. Disponível em: git-scm.com/book. Referência prática de versionamento para o desenvolvimento incremental do projeto.